

TITOLO

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione



UNIVERSITÀ POLITECNICA DELLE MARCHE

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

RELATORI:

Emanuele Storti

TESINA DI:

Tarek Naja
Sara Vaccaro

A.A 2024/2025

Indice

GLOSSARIO DEI TERMINI	2
GESTIONE DEI REQUISITI	4
Tabella MoSCoW dei Requisiti.	6
Diagrammi dei Casi d'Uso	7
Mockup	9
Verifica e Validazione del Software: Unit Tests	10
Test Manuali	10
Unit Test	10

Descrizione in linguaggio naturale

Il progetto consiste nello sviluppo di **Paws**, un'applicazione mobile pensata per amanti degli animali privati e rifugi. L'obiettivo dell'applicazione è offrire uno strumento per facilitare l'adozione e la condivisione di annunci relativi a cuccioli ed animali domestici, tenendo traccia delle schede di dettaglio dei singoli animali e consentendo il contatto diretto tra gli utenti.

Il sistema prevede due macro-tipologie di utenti:

- **Utenti privati:** individui che utilizzano l'applicazione per consultare gli annunci disponibili, salvare gli animali preferiti nella propria sezione dedicata, pubblicare nuovi annunci di adozione, filtrare la ricerca per razza, età, sesso e tipologia di utente, contattare direttamente i proprietari tramite chiamata telefonica, nonché cercare altri utenti e rifugi per animali (animal shelters) per visionarne i profili e i relativi post.
- **Rifugi e canili (Animal shelters):** strutture ed organizzazioni dedicate che dispongono di funzionalità per la pubblicazione e gestione di annunci di adozione su più ampia scala, promuovendo la visibilità degli animali in cerca di una casa e gestendo il proprio profilo informativo.

Sintesi dell'Approccio

Per la realizzazione del sistema, lo sviluppo dell'applicazione mobile è stato suddiviso in due percorsi complementari:

- **Applicazione Android nativa:** sviluppata in Kotlin sfruttando l'architettura nativa Android e la libreria di componenti UI, garantendo elevate prestazioni, gesture fluide (swipe-to-favorite) ed integrazione sia con i servizi di sistema (come l'avvio delle chiamate) sia con servizi di backend esterni. Nello specifico, sono stati utilizzati Firebase per l'autenticazione e la memorizzazione delle credenziali degli utenti e Supabase per il caricamento delle immagini e l'archiviazione dei dati completi relativi ai cuccioli.
- **Applicazione clone cross-platform:** realizzata in Flutter per consentire la portabilità del codice a partire da un'unica codebase, replicando fedelmente le funzionalità, l'identità visiva e l'interfaccia dell'applicazione nativa.

Questo approccio parallelo consente di analizzare e confrontare i vantaggi dello sviluppo nativo in termini di integrazione con il sistema operativo e risposta delle gesture rispetto ai benefici legati alla velocità di sviluppo e riusabilità offerti dalle moderne tecnologie cross-platform

Glossario dei Termini

Il seguente glossario definisce in modo univoco i termini di business e tecnologici utilizzati nell'ambito del progetto per evitare ambiguità durante le fasi di analisi, progettazione e sviluppo.

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Utente Privato	Persona fisica registrata che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Privato, User
Animal Shelter	Struttura d'accoglienza o associazione no-profit che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Rifugio, Ente di adozione
Cucciolo	Entità principale dell'applicazione che rappresenta un animale pubblicato con foto, nome, età, sesso, tipo e descrizione	BUSINESS	Animale, Post, Pet, Puppy
Scheda Dettaglio	Schermata informativa dedicata ad un singolo cucciolo contenente tutti i dettagli dell'animale e i contatti del proprietario	BUSINESS	Dettagli Cucciolo, Puppy Details
Preferiti	Sezione dell'applicazione in cui l'utente raccoglie e salva i cuccioli a cui è interessato	BUSINESS	Saved, Mi Piace, Favorites
Notifica	Messaggio automatico inviato al proprietario di un annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai preferiti	BUSINESS	Avviso, Notification
Filtro di Ricerca	Strumento per restringere l'elenco degli animali visualizzati in base a parametri quali età, sesso, tipo di animale e tipo di utente	BUSINESS	Criterio di Selezione, Search Filter

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Account	Insieme delle credenziali di accesso (email e password) e dei dati personali che identificano un utente nel sistema	TECNICO	Profilo
Swipe to Favorite	Gesto di scorrimento orizzontale della scheda verso destra che consente di salvare un cucciolo nei preferiti	TECNICO	Gesture Swipe, Scorrimento Destra
Firebase	Servizio cloud utilizzato per la gestione della registrazione, dell'autenticazione sicura e della sessione degli utenti	TECNICO	Auth, Gestione Accessi
Cloud Firestore	Database NoSQL in tempo reale utilizzato per memorizzare i dati degli utenti, dei post, dei preferiti e delle notifiche	TECNICO	DB NoSQL, Firestore
Supabase	Servizio di Object Storage utilizzato per l'hosting e l'erogazione performante delle immagini del profilo e delle foto dei cuccioli	TECNICO	Storage Cloud

Gestione dei requisiti

Nella presente sezione si analizzano in modo approfondito i requisiti del sistema, procedendo alla loro suddivisione secondo le seguenti categorie:

- **Requisiti funzionali**, che definiscono le specifiche funzionalità che il sistema è tenuto a fornire.
- **Requisiti non funzionali**, che individuano i vincoli e le qualità che il sistema deve soddisfare, quali ad esempio l'usabilità, la sicurezza e le prestazioni.

Requisiti Funzionali

Area: Gestione utente e autenticazione

- **RF1: Sign up:** il sistema deve permettere la creazione di un account inserendo email, password, nome, cognome, città, provincia, CAP, numero di telefono, username e la selezione del tipo di account (Utente Privato o Animal Shelter).
- **RF2: Sign in:** il sistema deve consentire agli utenti registrati di accedere all'applicazione inserendo le proprie credenziali.
- **RF3: Gestione e modifica profilo:** il sistema deve permettere all'utente di visualizzare e modificare i propri dati personali, aggiornare la foto profilo, richiedere il reset della password via email e modificare la tipologia di account.
- **RF4: Logout:** il sistema deve consentire all'utente di effettuare un logout sicuro dalla sessione attiva.
- **RF5: Delete account:** il sistema deve permettere all'utente di eliminare definitivamente il proprio account.

Area: Gestione post

- **RF6: Creazione cucciolo:** il sistema deve permettere all'utente di caricare un nuovo annuncio specificando nome del cucciolo, tipo di animale (cane, gatto, uccello), sesso (maschio, femmina), età, didascalia descrittiva e foto dell'animale.
- **RF7: Modifica ed eliminazione cucciolo:** il sistema deve permettere all'autore dell'annuncio di modificare le informazioni inserite o di eliminare definitivamente il post e la relativa immagine memorizzata nel cloud.
- **RF8: Visualizzazione feed e scheda dettaglio:** il sistema deve mostrare il feed degli annunci pubblicati e consentire la visualizzazione delle informazioni complete dell'animale e di contatto del proprietario.

Area: Preferiti e interazioni

- **RF9: Aggiunta e rimozione dai preferiti:** il sistema deve permettere di aggiungere un cucciolo ai preferiti e di rimuoverlo premendo sul cuore nella sezione preferiti.
- **RF10: Calcolo dei mi piace:** il sistema deve calcolare e mostrare in tempo reale il numero totale di salvataggi ricevuti per ciascun post.
- **RF11: Chiamata telefonica diretta:** il sistema deve consentire di avviare una chiamata verso il numero di telefono del proprietario.

Area: Ricerca e filtri

- **RF12: Ricerca:** il sistema deve permettere di cercare utenti o cuccioli inserendo il testo nella barra di ricerca.
- **RF13: Filtraggio:** il sistema deve consentire il filtraggio degli annunci in base a età, sesso, tipo di animale e tipo di utente.

Area: Notifiche

- **RF14: Generazione notifica:** il sistema deve generare una notifica destinata al proprietario dell'annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai/dai preferiti.
- **RF15: Consultazione e pulizia notifiche:** il sistema deve mostrare un elenco delle notifiche ricevute e permetterne l'eliminazione.

Requisiti Non Funzionali

Area: Prestazioni e Reattività

- **RNF1: Tempi di Risposta:** Il caricamento delle immagini e delle informazioni dal database cloud deve avvenire entro 2 secondi in condizioni normali di connettività.
- **RNF2: Fluidità delle Animazioni:** L'animazione del gesto Swipe to Favorite e il passaggio tra schermate devono mantenere una frequenza di aggiornamento fluida di 60 fps sia in Android nativo che in Flutter.

Area: Usabilità ed Esperienza Utente (UI/UX)

- **RNF3: Parità Visiva e Tipografica:** L'applicazione Flutter deve garantire una fedeltà visiva al 100% rispetto all'applicazione nativa Kotlin, rispettando la gerarchia dei colori (azzurro #78A7D3 per maschio, rosa #DC6FA6 per femmina) e dei font (serif, sans-serif-medium, font personalizzato Lacquer).
- **RNF4: Feedback all'Utente:** Ogni azione critica (salvataggio, eliminazione, invio notifiche) deve fornire un feedback visivo immediato tramite messaggi SnackBar o finestre di dialogo.

Area: Sicurezza e Gestione Dati

- **RNF5: Autenticazione e Cifratura:** La gestione delle credenziali di accesso deve avvenire in modo sicuro tramite protocolli cifrati gestiti da Firebase Authentication.
- **RNF6: Integrità e Pulizia dei Dati:** In caso di eliminazione di un utente o di un post, il sistema deve garantire la rimozione a cascata dei relativi file di immagine dallo storage cloud (Supabase Storage).

Area: Portabilità e Manutenibilità

- **RNF7: Multi-piattaforma:** La codebase Flutter deve essere completamente compatibile ed eseguibile sia su dispositivi Android che iOS senza modifiche alla logica di business.
- **RNF8: Modularità e Pulizia del Codice:** Il codice sorgente (sia Kotlin che Dart) deve essere strutturato in modo modulare separando i modelli dati, i servizi cloud ed i componenti di interfaccia utente.

Tabella MoSCoW dei Requisiti

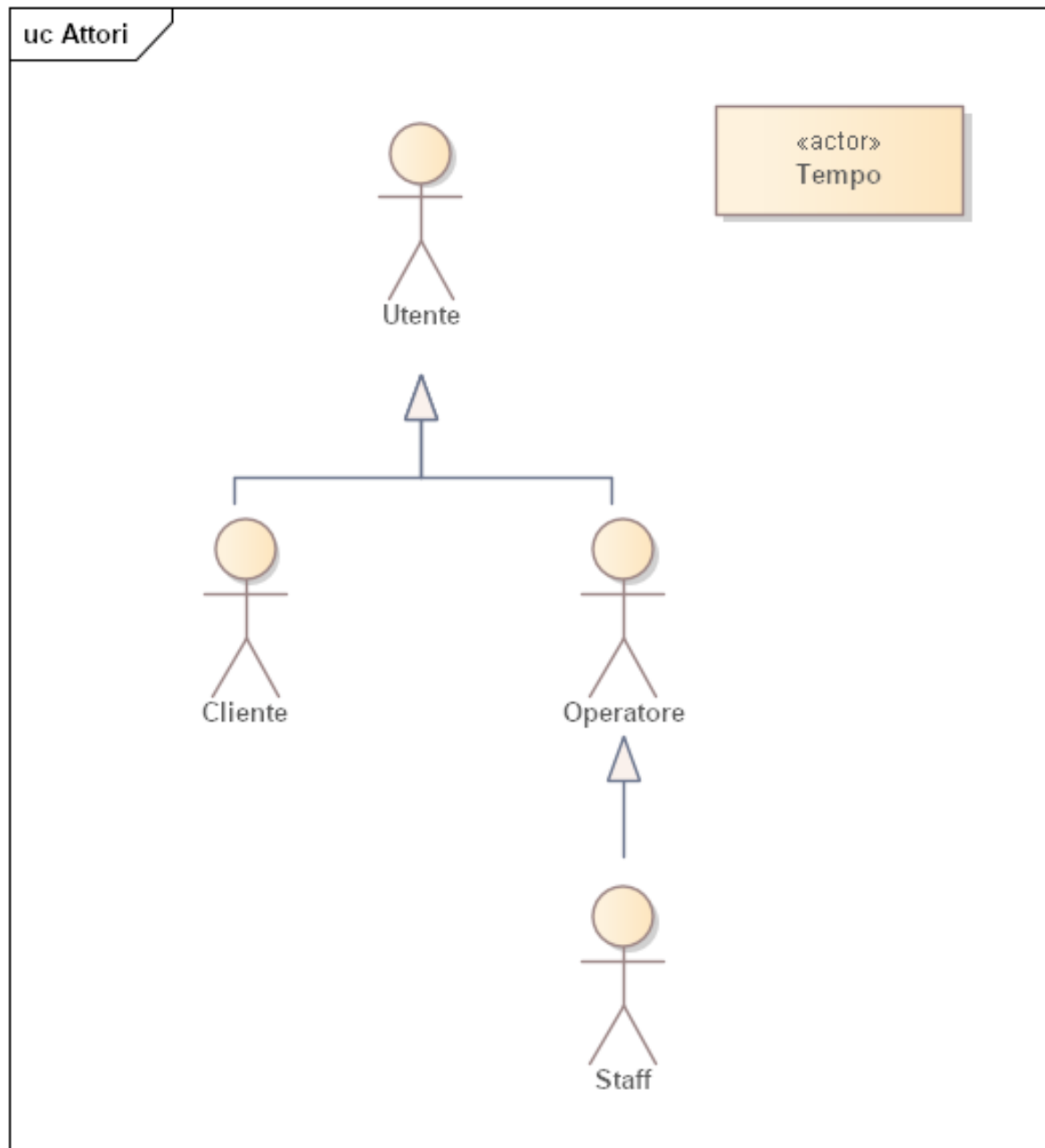
Di seguito la matrice MoSCoW che classifica i requisiti del sistema secondo le priorità: M (Must-have), S (Should-have), C (Could-have), W (Want-have).

Area	Requisito	Priorità MoSCoW
Gestione Utente	RF1: Registrazione Utente	Must
Gestione Utente	RF2: Login Utente	Must

Diagrammi dei Casi d'Uso

Diagramma degli Attori

Il diagramma individua gli attori coinvolti nel sistema e visualizza le connessioni di ereditarietà tra di essi.



Caso d'uso: RegistrazioneUtente	
ID	1
Breve descrizione	Lo Staff o il Cliente si registra nel sistema, con un ruolo che dipende dal punto di accesso
Attori primari	Cliente, Staff
Attori secondari	Nessuno
Precondizioni	Nessuna
Sequenza degli eventi principale	1. If l'Utente è un Cliente che accede alla pagina di registrazione pubblica, allora 1.1. Il sistema mostra il modulo di registrazione 1.2. Il Cliente inserisce le proprie informazioni (nome, cognome, email, password) 1.3. While le credenziali inserite del Cliente non sono valide 1.1. Il sistema richiede al Cliente di inserire le sue informazioni 1.4. Il sistema valida i dati inseriti 1.5. Il sistema crea un nuovo account con il ruolo di Cliente
Postcondizioni	1. È stato creato un nuovo account Utente con ruolo Cliente
Sequenza degli eventi alternativa	1. If l'Utente è un membro dello Staff che accede alla pagina di gestione utenti interna, allora 1.1. Il sistema mostra un modulo di creazione Utente 1.2. Il membro dello Staff inserisce le informazioni del nuovo Utente (nome, cognome, email, password) e seleziona il ruolo desiderato (Operatore o Staff) 1.3. While le credenziali inserite dell'Utente non sono valide 1.1. Il sistema richiede allo Staff di inserire le sue informazioni 1.4. Il sistema valida i dati inseriti 1.5. Il sistema crea un nuovo account con il ruolo specificato (Operatore o Staff)

Mockup

Il mockup dell'applicativo GESTRi è stato realizzato con Figma, uno strumento per la progettazione e la prototipazione di interfacce utente. Ogni mockup rappresenta la versione progettuale dell'interfaccia, definendo struttura, elementi grafici e funzionalità prima della fase di sviluppo.



Figura 1: Mockup login

Verifica e Validazione del Software: Unit Tests

I seguenti test rappresentano l'insieme delle attività di verifica implementate per garantire il corretto funzionamento del progetto. Sono stati adottati due approcci complementari: test automatici lato codice e test manuali delle API.

Test Manuali

Per verificare le API è stata utilizzata l'applicazione Postman, uno strumento che consente di inviare richieste HTTP e analizzare le risposte. Questi test non rientrano nel diagramma degli Unit Test, ma rappresentano un'importante componente di validazione dell'integrazione e del comportamento delle API lato client-server.

Unit Test

La sezione presenta i test unitari eseguiti sul modulo Attività di GESTRi, con l'obiettivo di verificarne la correttezza funzionale e assicurare l'affidabilità del relativo codice.

```
1 class AttivitaIntegrationTests(TestCase):
2     def setUp(self):
3         self.client = Client()
4         # create users
5         self.staff = Utente.objects.create(email='staff@example.com', ruolo=
            Ruolo.STAFF)
6         self.staff.set_password('pass')
7         self.staff.save()
```

Unit Test 1: Attivita